GLOBAL SOLUTION 2026/2 • TEMA: CITIZENTECH • TURMA: 2TDSPF

Fix-It — Mastering Relational & Non-Relational Database

Documentação Técnica Integrada • Persistência Poliglota • Evidências de Execução

1. Integrantes do Grupo

Nome Completo do Aluno	Registro de Matrícula (RM)	Turma
Gabriel Sbrana Campos	565849	2TDSPF
Moisés Waidemann Molinillo	563719	2TDSPF
Richard Freitas	566127	2TDSPF
Thiago Rodrigues da Mota	563650	2TDSPF

2. Visão Geral da Solução (Problema CitizenTech)

O **Fix-It** é uma plataforma colaborativa de zeladoria urbana voltada para identificação, registro georreferenciado e priorização inteligente de incidentes viários (buracos no asfalto, falhas de iluminação pública, bueiros obstruídos e descarte irregular de resíduos). A solução transforma o cidadão em um sensor urbano ativo e combate a morosidade e a falta de transparência da administração pública municipal.

Para atender aos requisitos de alta performance, auditoria de dados públicos e inteligência analítica, o sistema implementa uma **arquitetura de persistência poliglota**:

- **Oracle Database (Relacional):** Responsável pela consistência transacional estrita (ACID), trilha de auditoria completa das transições de status operacional, aplicação de integridade referencial e execução de rotinas programadas via PL/SQL.
- **MongoDB (Orientado a Documentos):** Otimizado para consultas geoespaciais instantâneas em raio de proximidade (índice `2dsphere`) e armazenamento desnormalizado dos relatórios com metadados de Inteligência Artificial e ordens de serviço.

3. Estrutura do Banco Relacional (Oracle PL/SQL)

O arquivo `script-bd-oracle.sql` estrutura e implementa os componentes relacionais da solução:

- **Tabelas Relacionais:** `TB_USUARIO` (cidadãos e fiscais municipais), `TB_OCORRENCIA` (incidentes registrados), `TB_ORDEM_SERVICO` (execução de reparos e equipes) e `TB_AUDITORIA_OCORRENCIA` (trilha de logs históricos).
- **Gatilhos (Triggers):**
  - `TRG_AUDITORIA_OCORRENCIA`: Gatilho de auditoria a nível de linha disparado após `UPDATE`, registrando valores antigos e novos de status e gravidade, data/hora e o usuário do banco.
  - `TRG_VALIDA_ABERTURA_OCORRENCIA`: Gatilho de validação disparado antes de `INSERT` para impedir datas futuras ou cadastro de ocorrências já concluídas.

- **Pacotes PL/SQL (Packages):**
  - `PKG_FIXIT_OCORRENCIAS`: Reúne a função `FN_CALCULAR_INDICE_RESOLUCAO` (calcula o percentual de resolução por bairro) e a procedure `PR_ESCALAR_GRAVIDADE_ATRASADA` (escalona para nível urgente chamados abertos acima de X dias).
  - `PKG_FIXIT_AUDITORIA`: Reúne a função `FN_TEMPO_MEDIO_ATENDIMENTO` (tempo médio em dias para reparo por categoria) e a procedure `PR_FINALIZAR_OCORRENCIA` (conclui o incidente e registra a ordem de serviço associada).
- **Blocos Anônimos:** Rotinas que utilizam cursores explícitos com estruturas de decisão para triagem de criticidade e execução integrada de testes de homologação dos pacotes.

4. Estrutura do Banco Não-Relacional (MongoDB)

O arquivo `script-mongo-fixit.js` opera sobre o banco de dados `zeladoria_fixit`:

- **Coleções Criadas:** `usuarios_cidadaos` e `ocorrencias_urbanas` (contendo 11 documentos cadastrados, superando o requisito mínimo de 10).
- **Indexação Otimizada:** Criação de índice geoespacial `2dsphere` no campo `localizacao` (estrutura GeoJSON) e índice composto em `{ status: 1, categoria: 1 }`.
- **Decisão de Modelagem de Dados:**
  - *Incorporação (Embedding):* Os metadados de Inteligência Artificial (`ia_analise`) e os dados de despacho da equipe operacional (`ordem_servico`) foram embutidos no documento da ocorrência. Essa abordagem elimina custos computacionais de junção (`JOIN`) e acelera a renderização no aplicativo móvel.
  - *Referência (Referencing):* O munícipe solicitante é referenciado pelo campo `id_usuario_referencia` (apontando para `usuarios_cidadaos`), prevenindo a duplicação de informações sensíveis de contato e atendendo às diretrizes da LGPD.

5. Mapeamento & Consultas Comparativas

Abaixo está detalhada a correspondência de modelagem entre os dois paradigmas:

Conceito de Negócio	Oracle Database (Relacional)	MongoDB (Documentos)	Estratégia Adotada
Identificação	`id_ocorrencia (NUMBER IDENTITY)`	`_id (ObjectId) / id_oracle_ref`	Chave primária mapeada para BSON ID único
Ponto Geográfico	Colunas `latitude` e `longitude`	Objeto GeoJSON `Point [lng, lat]`	Transformação para formato nativo espacial
Auditoria / Análise IA	Tabela `TB_AUDITORIA_OCORRENCIA`	Subdocumento embutido `ia_analise`	Incorporação direta para alta velocidade de leitura
Ordem de Reparo	Tabela filha `TB_ORDEM_SERVICO`	Subdocumento embutido `ordem_servico`	Consolidação 1:1 incorporada no chamado

Conceito de Negócio	Oracle Database (Relacional)	MongoDB (Documentos)	Estratégia Adotada
Munícipe Solicitante	Chave Estrangeira `id_usuario (FK)`	Referência `id_usuario_referencia`	Referenciamento desnormalizado entre coleções

Relatório 1 — Agrupamento por Categoria e Status (Agregação)

Consolidação do volume total de chamados abertos e concluídos por tipologia de anomalia viária.

```
-- Oracle SQL SELECT categoria, status, COUNT(*) AS total_chamados FROM TB_OCORRENCIA GROUP BY categoria, status ORDER BY categoria, status;
```

```
// MongoDB Aggregation Pipeline db.ocorrencias_urbanas.aggregate([ { $group: { _id: { categoria: "$categoria", status: "$status" }, total_chamados: { $sum: 1 } } }, { $sort: { "_id.categoria": 1, "_id.status": 1 } } ]);
```

Relatório 2 — Chamados Críticos Pendentes (Filtro e Projeção)

Triagem de ocorrências de severidade alta ou urgente que exigem intervenção operacional imediata.

```
-- Oracle SQL SELECT id_ocorrencia, categoria, bairro, gravidade, TO_CHAR(data_abertura, 'YYYY-MM-DD') AS data_abertura FROM TB_OCORRENCIA WHERE gravidade IN ('ALTA', 'URGENTE') AND status <> 'CONCLUIDO';
```

```
// MongoDB Find com Projeção db.ocorrencias_urbanas.find( { gravidade: { $in: ["ALTA", "URGENTE"] }, status: { $ne: "CONCLUIDO" } }, { categoria: 1, bairro: 1, gravidade: 1, data_abertura: 1, _id: 1 } );
```

Relatório 3 — Busca Geoespacial em Raio de 1 km (Proximidade)

Identificação de ocorrências no entorno imediato da Avenida Paulista (-23.561684, -46.655981).

```
-- Oracle SQL (Busca por Delimitação Euclidiana de Coordenadas) SELECT id_ocorrencia, categoria, bairro, latitude, longitude FROM TB_OCORRENCIA WHERE latitude BETWEEN -23.570000 AND -23.550000 AND longitude BETWEEN -46.665000 AND -46.645000;
```

```
// MongoDB Operador Geoespacial $near Nativo db.ocorrencias_urbanas.find({ localizacao: { $near: { $geometry: { type: "Point", coordinates: [-46.655981, -23.561684] }, $maxDistance: 1000 } } }, { categoria: 1, bairro: 1, status: 1, localizacao: 1 });
```

6. Evidências Fotográficas de Homologação

As capturas de tela a seguir comprovam a execução prática e a integridade de todos os scripts e objetos.

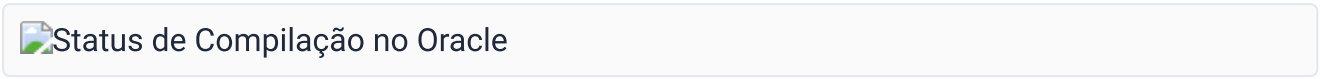

Status de Compilação no Oracle

Figura 1 — Compilação com status VALID dos pacotes PKG_FIXIT_OCORRENCIAS, PKG_FIXIT_AUDITORIA e triggers no Oracle.

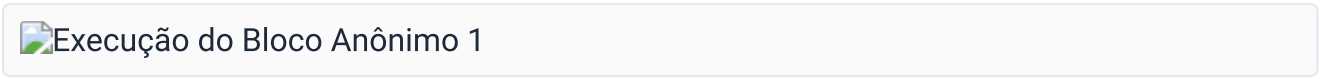

Execução do Bloco Anônimo 1

Figura 2 — Saída do Bloco Anônimo 1 exibindo triagem de ocorrências críticas com cursor explícito.

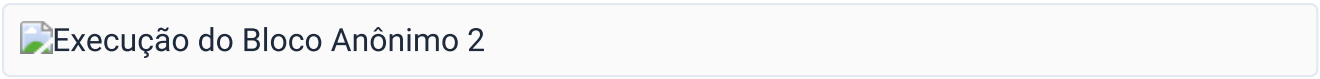

Execução do Bloco Anônimo 2

Figura 3 — Saída do Bloco Anônimo 2 demonstrando a execução das rotinas dos pacotes de negócio e auditoria.


Tabela de Auditoria Preenchida pelo Gatilho

Figura 4 — Registro gerado na tabela TB_AUDITORIA_OCORRENCIA pelo gatilho TRG_AUDITORIA_OCORRENCIA.

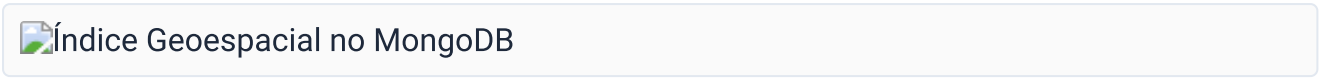

Índice Geoespacial no MongoDB

Figura 5 — Índice localizacao_2dsphere (GEOSPATIAL) ativo com status READY no MongoDB Compass.

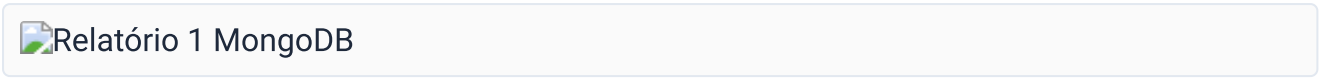

Relatório 1 MongoDB

Figura 6 — Resultado do Relatório 1 (Agregação $group por categoria e status) no terminal mongosh.

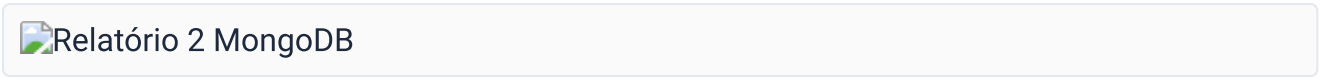

Relatório 2 MongoDB

Figura 7 — Resultado do Relatório 2 (Consulta com projeção de chamados críticos pendentes) no mongosh.

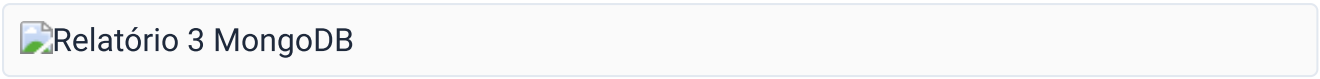

Relatório 3 MongoDB

Figura 8 — Resultado do Relatório 3 (Consulta de raio de 1 km com operador $near) no mongosh.

7. Instruções de Reprodução e Execução

Ambiente Oracle Database

1. Conecte-se ao seu schema utilizando o Oracle SQL Developer ou a ferramenta SQLcl.
2. Abra o arquivo `script-bd-oracle.sql` e execute todo o script pressionando **F5** (Executar como Script).
3. Habilite a visualização de saída do console via comando `SET SERVEROUTPUT ON;` para acompanhar os logs dos blocos anônimos.

Ambiente MongoDB

1. Certifique-se de que o daemon local do MongoDB está em execução ou estabeleça conexão com o MongoDB Atlas.
2. No MongoDB Shell (`mongosh`) ou terminal integrado do MongoDB Compass, execute:

```
mongosh < script-mongo-fix-it.js
```

3. Navegue pelas coleções `usuarios_cidadaos` e `ocorrencias_urbanas` para auditar os índices e documentos criados.